

LLM Transmission 8GB — Comprehensive Findings Report

Generated: September 02, 2026

NVIDIA Jetson Nano 8GB | llama.cpp CUDA | 27 models benchmarked

This report documents the complete benchmark findings across 27 local LLMs, the ideal settings for each model, and the methodology used to create the LLM Transmission routing service.

Table of Contents

1. Methodology — How We Benchmarked 27 Models
2. Summary Leaderboard — All 27 Models Ranked
3. Temperature Sweep Findings
4. Top-K Sweep Findings
5. DeepSeek Template Discovery
6. Per-Model Findings (27 models, each with ideal settings)
7. Category Analysis — Coding, Creative, Function Calls
8. How We Built the Transmission Service
9. Routing Table — Top 6 Models Per Category
10. Conclusions and Next Steps

1. Methodology — How We Benchmarked 27 Models

Hardware

All tests run on NVIDIA Jetson Nano 8GB using llama.cpp with CUDA acceleration (-ngl 999 -fa on). The GUI (gdm3) was disabled during benchmarking to maximize available RAM for model inference.

Prompt Suite — 12 Prompts Across 6 Categories

Scoring System

Each prompt was scored on a 0-10 scale based on task-specific criteria. Scores were assigned by analyzing the model output against expected structure, correctness, completeness, and format compliance. For function calls, the key criterion was producing valid JSON tool-call format (not prose descriptions). For poetry, strict iambic pentameter meter and correct rhyme scheme were required for high scores.

Sweep Methodology

Each model was tested across multiple parameter sweeps, one variable at a time. Quality was always evaluated before speed — a model that produces wrong answers quickly is not useful.

Standard Parameters (held constant)

top_p = 0.9 | repeat_penalty = 1.1 | max_tokens = 2048 | Context = 16384 (32k for non-thinking models where VRAM allows) | chat template = per-model (chatml for most, gemma for Gemma models, chatml for DeepSeek R1 distills)

2. Summary Leaderboard — All 27 Models Ranked

Overall score is the average across all 12 prompts at each model's best temperature with k=40.
Group scores are averages within each category.

3. Temperature Sweep Findings

Each of the 27 models was tested at 7 temperatures (0.0, 0.1, 0.2, 0.3, 0.5, 0.7, 1.0) with all other parameters held constant. The best temperature was locked per model based on highest average score across 12 prompts.

Key insight: There is no single 'best' temperature. 7 models peak at $t=0.3$ and 7 at $t=1.0$, with the rest distributed across the range. Reasoning/thinking models (DeepSeek R1, Ministral Reasoning) tend to prefer low temperatures (0.0-0.3), while creative models (Granite 4.1, Qwen 3 1.7B) benefit from high temperatures (1.0). This is why the transmission service stores a per-model locked temperature rather than using a global default.

4. Top-K Sweep Findings

After locking the best temperature per model, we swept top_k across 3 values: 20, 40, and 64. This produced $27 \times 3 \times 12 = 972$ additional scored runs.

k=40 is the confirmed default for the transmission service. The 5 models that prefer k=20 are still tested at k=40 in the routing table (the difference is typically <0.3 points), and k=40 provides the best balance across all categories.

5. DeepSeek Template Discovery

During benchmarking, DeepSeek R1-Distill models (1.5B and 7B) scored remarkably low — 3.7 and 2.4 respectively. Investigation revealed the models were ignoring prompts entirely, hallucinating about 'DeepSeek company' instead of answering questions.

Root cause: The eval harness was using the 'deepseek' chat template, which is designed for the original DeepSeek Coder models. However, DeepSeek R1-Distill models are Qwen-architecture models fine-tuned by DeepSeek — they use the ChatML format (<|im_start|>/<|im_end|> tags), not the DeepSeek-specific format.

Before Fix (deepseek template)

After Fix (chatml template)

Lesson: Always verify the chat template matches the model's architecture, not just its name. A 'DeepSeek' model may not use the 'deepseek' template. This finding is encoded in the transmission service's routing table — both DeepSeek models use 'chatml' and the test suite includes a specific assertion (test_deepseek_uses_chatml_not_deepseek) to prevent regression.

6. Per-Model Findings — 27 Models with Ideal Settings

Each model below includes its benchmark scores by category, ideal settings, per-prompt breakdown, and notable observations from the benchmark.

#1 — hermes3-3b-q5

Overall: 7.8/10 | Coding/Math: 9.8 | Creative/Poetry: 9.0 | Function Calls: 6.2 | Speed: 19.4 tok/s

- Excellent at coding and math tasks
- Excellent at creative writing and poetry
- Recommended for production use

#2 — smallthinker-3b

Overall: 7.6/10 | Coding/Math: 8.5 | Creative/Poetry: 5.5 | Function Calls: 7.7 | Speed: 20.1 tok/s

- Strong function-call format compliance
- Reasoning model — produces thinking blocks before answers, needs --jinja flag
- Recommended for production use

#3 — deepseek-r1-7b

Overall: 7.5/10 | Coding/Math: 10.0 | Creative/Poetry: 7.0 | Function Calls: 6.3 | Speed: 9.7 tok/s

- Excellent at coding and math tasks
- Slow generation (9.7 tok/s) — may timeout on long prompts
- Reasoning model — produces thinking blocks before answers, needs --jinja flag
- Recommended for production use

#4 — hermes3-3b-q4

Overall: 7.5/10 | Coding/Math: 9.8 | Creative/Poetry: 6.5 | Function Calls: 6.3 | Speed: 19.7 tok/s

- Excellent at coding and math tasks
- Recommended for production use

#5 — qwen2.5-3b

Overall: 7.5/10 | Coding/Math: 9.0 | Creative/Poetry: 8.5 | Function Calls: 6.2 | Speed: 20.1 tok/s

- Excellent at coding and math tasks
- Recommended for production use

#6 — qwen3.5-2b

Overall: 7.5/10 | Coding/Math: 9.8 | Creative/Poetry: 9.0 | Function Calls: 5.5 | Speed: 24.9 tok/s

- Excellent at coding and math tasks
- Excellent at creative writing and poetry
- Reasoning model — produces thinking blocks before answers, needs --jinja flag
- Recommended for production use

#7 — granite4.1-3b

Overall: 7.4/10 | Coding/Math: 9.2 | Creative/Poetry: 8.5 | Function Calls: 5.8 | Speed: 18.9 tok/s

- Excellent at coding and math tasks
- Recommended for production use

#8 — ministral-3b-reasoning

Overall: 7.2/10 | Coding/Math: 9.5 | Creative/Poetry: 9.5 | Function Calls: 5.0 | Speed: 18.7 tok/s

- Excellent at coding and math tasks
- Excellent at creative writing and poetry
- Reasoning model — produces thinking blocks before answers, needs --jinja flag
- Recommended for production use

#9 — qwen3-1.7b

Overall: 7.2/10 | Coding/Math: 9.8 | Creative/Poetry: 5.5 | Function Calls: 6.0 | Speed: 30.8 tok/s

- Excellent at coding and math tasks
- Fast generation (30.8 tok/s)
- Recommended for production use

#10 — smollm3

Overall: 7.2/10 | Coding/Math: 9.2 | Creative/Poetry: 7.0 | Function Calls: 5.8 | Speed: 20.8 tok/s

- Excellent at coding and math tasks
- Recommended for production use

#11 — granite3.2-2b

Overall: 7.0/10 | Coding/Math: 9.5 | Creative/Poetry: 7.0 | Function Calls: 5.3 | Speed: 26.6 tok/s

- Excellent at coding and math tasks
- Fast generation (26.6 tok/s)
- Recommended for production use

#12 — granite4.2-3b

Overall: 6.9/10 | Coding/Math: 9.5 | Creative/Poetry: 6.5 | Function Calls: 5.3 | Speed: 18.9 tok/s

- Excellent at coding and math tasks

#13 — qwen2.5-coder-3b

Overall: 6.9/10 | Coding/Math: 9.5 | Creative/Poetry: 8.5 | Function Calls: 4.7 | Speed: 20.0 tok/s

- Excellent at coding and math tasks

#14 — granite4-3b

Overall: 6.8/10 | Coding/Math: 9.5 | Creative/Poetry: 6.5 | Function Calls: 5.0 | Speed: 18.9 tok/s

- Excellent at coding and math tasks

#15 — stablelm-zephyr

Overall: 6.8/10 | Coding/Math: 9.8 | Creative/Poetry: 8.0 | Function Calls: 4.3 | Speed: 26.4 tok/s

- Excellent at coding and math tasks
- Fast generation (26.4 tok/s)

#16 — lfm2.5-2.6b

Overall: 6.6/10 | Coding/Math: 10.0 | Creative/Poetry: 5.5 | Function Calls: 4.7 | Speed: 23.8 tok/s

- Excellent at coding and math tasks

#17 — deepseek-r1-1.5b

Overall: 5.3/10 | Coding/Math: 9.2 | Creative/Poetry: 4.5 | Function Calls: 3.0 | Speed: 32.7 tok/s

- Excellent at coding and math tasks
- Struggles with function-call JSON format — produces prose instead
- Fast generation (32.7 tok/s)
- Reasoning model — produces thinking blocks before answers, needs --jinja flag

#18 — ministral-3b

Overall: 4.9/10 | Coding/Math: 7.2 | Creative/Poetry: 5.5 | Function Calls: 3.2 | Speed: 24.7 tok/s

- Struggles with function-call JSON format — produces prose instead

#19 — gemma2-2b

Overall: 4.5/10 | Coding/Math: 2.8 | Creative/Poetry: 4.0 | Function Calls: 5.8 | Speed: 22.3 tok/s

#20 — llama3.2-3b

Overall: 2.8/10 | Coding/Math: 1.8 | Creative/Poetry: 5.0 | Function Calls: 2.7 | Speed: 19.7 tok/s

- Struggles with function-call JSON format — produces prose instead
- Very poor overall — not recommended for production use

#21 — gemma3-1b

Overall: 2.7/10 | Coding/Math: 2.2 | Creative/Poetry: 3.5 | Function Calls: 2.7 | Speed: 31.5 tok/s

- Struggles with function-call JSON format — produces prose instead
- Fast generation (31.5 tok/s)
- Very poor overall — not recommended for production use

#22 — phi3-3.8b

Overall: 2.5/10 | Coding/Math: 2.2 | Creative/Poetry: 4.0 | Function Calls: 2.2 | Speed: 17.7 tok/s

- Struggles with function-call JSON format — produces prose instead
- Very poor overall — not recommended for production use

#23 — llama3.2-1b

Overall: 2.4/10 | Coding/Math: 1.2 | Creative/Poetry: 4.5 | Function Calls: 2.5 | Speed: 42.2 tok/s

- Struggles with function-call JSON format — produces prose instead
- Fast generation (42.2 tok/s)
- Very poor overall — not recommended for production use

#24 — phi4-mini

Overall: 2.3/10 | Coding/Math: 2.5 | Creative/Poetry: 3.0 | Function Calls: 2.0 | Speed: 16.8 tok/s

- Struggles with function-call JSON format — produces prose instead
- Reasoning model — produces thinking blocks before answers, needs --jinja flag
- Very poor overall — not recommended for production use

#25 — gemma3n-e2b

Overall: 2.2/10 | Coding/Math: 1.5 | Creative/Poetry: 3.5 | Function Calls: 2.3 | Speed: 21.1 tok/s

- Struggles with function-call JSON format — produces prose instead
- Reasoning model — produces thinking blocks before answers, needs --jinja flag
- Very poor overall — not recommended for production use

#26 — llama3.2-3b-new

Overall: 2.2/10 | Coding/Math: 1.5 | Creative/Poetry: 3.5 | Function Calls: 2.2 | Speed: 19.7 tok/s

- Struggles with function-call JSON format — produces prose instead
- Very poor overall — not recommended for production use

#27 — gemma4-e2b

Overall: 1.2/10 | Coding/Math: 0.0 | Creative/Poetry: 3.5 | Function Calls: 1.3 | Speed: 19.8 tok/s

- Struggles with function-call JSON format — produces prose instead
- Reasoning model — produces thinking blocks before answers, needs --jinja flag
- Very poor overall — not recommended for production use

7. Category Analysis

Coding & Math — Best Models

Creative & Poetry — Best Models

Function Calls — Best Models

8. How We Built the Transmission Service

Overview

The LLM Transmission 8GB service is a local-first prompt routing system. It classifies incoming prompts, presents the top 6 model options to the operator, then runs the prompt with the selected model using its benchmark-optimized settings. The goal: handle as many tasks as possible on free local LLMs, minimizing cloud token usage.

Step 1: Benchmark Data Collection

We tested 27 models from the jetson-model-zoo project. Each model ran a suite of 12 prompts spanning 6 categories (HTML, Python, Math, Poetry, Creative Writing, Function Calls). We swept temperature across 7 values and top_k across 3 values, producing 2,916 total scored rows. All scores were recorded in a CSV and used to determine the best parameter combination for each model.

Step 2: Parameter Optimization (One Variable at a Time)

Following the principle 'quality before speed,' we optimized one parameter at a time: 1. Temperature sweep: 7 values (0.0-1.0), all models, all prompts. Best temp locked per model. 2. Top-K sweep: 3 values (20, 40, 64) at locked temps. k=40 confirmed optimal for 70% of models. 3. Template verification: Discovered DeepSeek R1 distills need chatml, not deepseek. Fixed and re-benchmarked. Next planned sweep: top_p (not yet conducted as of this report).

Step 3: Prompt Classification

The classifier uses keyword matching and heuristics — no LLM is needed for classification itself, making it instant (<1ms). It scores each prompt against keyword indices for 4 categories: • coding_math: Python, HTML, math proofs (keywords: python, def, html, flexbox, prove, proof) • creative_poetry: Creative writing, poetry (keywords: poem, iambic, pentameter, story, fiction) • function_calls: Tool-use format (keywords: terminal, sqlite, email, web search, read file) • general_purpose: Fallback for questions and explanations The classifier also applies heuristic boosts (e.g., 'write a function' without a language name still triggers coding_math).

Step 4: Routing Table Construction

For each category, the top 6 models were selected from the benchmark data, ranked by their group-average quality score. Each model entry in the routing table includes: • Quality score (group-specific average, 1-10 scale) • Generation speed (tokens/second) • Model file size (GB) • Chat template (chatml, gemma, etc.) • Thinking flag (requires --jinja) • Best temperature (locked from sweep) • Top-K (locked at 40) • Model file path

Step 5: Operator Presentation and Execution

When a prompt is submitted, the service: 1. Classifies the prompt (<1ms) 2. Presents the top 6 models with quality, speed, size, thinking flag, and a Q×S (quality × speed) metric 3. Operator selects a model (or accepts #1 default) 4. Builds and executes a llama-cli command with the model's best settings 5. Cleans the output (removes banner, spinner characters, stats) 6. Reports results: model used, category, time, tokens generated, cloud tokens saved

9. Routing Table — Top 6 Models Per Category

Coding Math

Python code, HTML pages, math proofs — structured generation requiring precision

Creative Poetry

Creative writing, poetry, iambic pentameter, stories — requires linguistic artistry

Function Calls

Hermes tool calls: terminal, write_file, read_file, web_search, sqlite, email — structured tool-use format

General Purpose

General-purpose prompts, questions, explanations — fallback for unclassified prompts

10. Conclusions and Next Steps

Key Findings

- hermes3-3b-q5 is the overall champion (7.8/10) — strong across all categories, the best general-purpose model.
- lfm2.5-2.6b achieves perfect coding & math scores (10.0/10) at 23.8 tok/s — the best coding specialist.
- ministral-3b-reasoning is the best creative writer (9.5/10) but slow at 18.7 tok/s.
- smallthinker-3b is the best function caller (7.7/10) — the only model that reliably produces valid tool-call JSON.
- DeepSeek R1 distills need chatml template, not deepseek — using the wrong template caused 43-158% score degradation.
- k=40 is the universal optimal top_k (70% of models) — safe default for the routing table.
- No single temperature works for all models — 7 peak at 0.3, 7 at 1.0. Per-model locked temps are essential.
- Function calls are the hardest category — no model scores above 7.7/10. Most models produce prose instead of structured JSON.
- Models under 2B parameters (gemma3-1b, llama3.2-1b, phi3-3.8b) score below 3.0/10 — not suitable for production use.
- The Q×S metric (quality × speed / 10) helps operators balance quality vs speed when selecting a model.

Next Steps

- Top-P sweep: Test top_p values (0.8, 0.85, 0.9, 0.95, 1.0) at locked temps and k=40.
- Repeat penalty sweep: Test values 1.0, 1.05, 1.1, 1.15, 1.2 to find optimal per-model settings.
- Context window optimization: Test 8K vs 16K vs 32K context to find the sweet spot for 8GB VRAM.
- Function-call fine-tuning: Most models struggle with tool-call JSON — explore system prompts or fine-tuning.
- Add more models: Test new releases as they become available (e.g., Qwen 3.5, Granite 5).
- Web interface: Add a Flask web UI for non-CLI usage.
- Auto-routing mode: When operator doesn't want to choose, auto-select #1 based on quality score.